



INSTITUTO POLITÉCNICO NACIONAL

UPIICSA

ABSTRACCIÓN Y USO DE DATOS

EQUIPO: SYNTAX ERROR

3CM32

REPORTE PRÁCTICA 18

Desarrollo: Quick sort para ordenar conjunto de caracteres, enteros y nuevo tipo de dato en PE y POO utilizando la técnica de partición y recursividad.

Errores:

- Error 1: Recursión infinita provocada por una mala elección del pivote o por omitir la condición base de parada.
- Error 2: Pasar un índice superior incorrecto al invocar la función recursiva inicial (usar el tamaño total N en lugar de $N - 1$).

Soluciones:

- Solución 1: Validar siempre los límites con la condición base if (bajo < alto) antes de efectuar las llamadas recursivas.
- Solución 2: Asegurar que el límite superior enviado inicialmente corresponda al último índice válido del arreglo (n - 1).

Código:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef int  (*Comparador)(const void *a, const void *b);
typedef void (*Impresor)(const void *elemento);

typedef struct Ordenador Ordenador;
struct Ordenador {
    void      *datos;
    int        n;
    size_t     tam;
    Comparador comparar;
    Impresor   imprimir;
    void (*ordenar)(Ordenador *self);
    void (*mostrar)(const Ordenador *self, const char *titulo);
};

static void *Ordenador_elem(const Ordenador *self, int i) {
    return (char *)self->datos + (size_t)i * self->tam;
}
```

```c
static void Ordenador_intercambiar(Ordenador *self, int i, int j) {
    if (i == j) return;
    char *a = (char *)Ordenador_elem(self, i);
    char *b = (char *)Ordenador_elem(self, j);
    for (size_t k = 0; k < self->tam; k++) {
        char t = a[k]; a[k] = b[k]; b[k] = t;
    }
}

static int Ordenador_particion(Ordenador *self, int bajo, int alto) {
    void *pivote = Ordenador_elem(self, alto);
    int i = bajo - 1;
    for (int j = bajo; j < alto; j++) {
        if (self->comparar(Ordenador_elem(self, j), pivote) <= 0) {
            i++;
            Ordenador_intercambiar(self, i, j);
        }
    }
    Ordenador_intercambiar(self, i + 1, alto);
    return i + 1;
}

static void Ordenador_quicksort(Ordenador *self, int bajo, int alto) {
    if (bajo < alto) {
        int p = Ordenador_particion(self, bajo, alto);
        Ordenador_quicksort(self, bajo, p - 1);
        Ordenador_quicksort(self, p + 1, alto);
    }
}

static void Ordenador_ordenar(Ordenador *self) {
    if (self->n > 1)
        Ordenador_quicksort(self, 0, self->n - 1);
}

static void Ordenador_mostrar(const Ordenador *self, const char *titulo) {
    printf("%s", titulo);
    for (int i = 0; i < self->n; i++)
        self->imprimir(Ordenador_elem(self, i));
    printf("\n");
}

Ordenador *Ordenador_nuevo(void *datos, int n, size_t tam,
                           Comparador cmp, Impresor imp) {
    Ordenador *o = (Ordenador *)malloc(sizeof(Ordenador));
    if (!o) return NULL;
    o->datos    = datos;
    o->n        = n;
```

```c
    o->tam      = tam;
    o->comparar = cmp;
    o->imprimir = imp;
    o->ordenar  = Ordenador_ordenar;
    o->mostrar  = Ordenador_mostrar;
    return o;
}

void Ordenador_destruir(Ordenador *o) {
    free(o);
}

int comparaInt(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}
void imprimeInt(const void *e) { printf("%d ", *(const int *)e); }

int comparaChar(const void *a, const void *b) {
    char x = *(const char *)a, y = *(const char *)b;
    return (x > y) - (x < y);
}
void imprimeChar(const void *e) { printf("%c ", *(const char *)e); }

typedef struct {
    char  nombre[30];
    int   matricula;
    float promedio;
} Alumno;

Alumno Alumno_nuevo(const char *nombre, int matricula, float promedio) {
    Alumno a;
    strncpy(a.nombre, nombre, sizeof(a.nombre) - 1);
    a.nombre[sizeof(a.nombre) - 1] = '\0';
    a.matricula = matricula;
    a.promedio  = promedio;
    return a;
}

int Alumno_comparar(const void *a, const void *b) {
    float x = ((const Alumno *)a)->promedio;
    float y = ((const Alumno *)b)->promedio;
    return (x > y) - (x < y);
}

void Alumno_imprimir(const void *e) {
    const Alumno *a = (const Alumno *)e;
    printf("\n  %-8s | %d | %.1f", a->nombre, a->matricula, a->promedio);
```

```c
}

int main(void) {
    int    nums[] = {34, 7, 23, 32, 5, 62, 32, 1};
    char   cars[] = {'m', 'z', 'a', 'k', 'b', 'x'};
    Alumno alu[]  = {
        Alumno_nuevo("Lisette",   1001, 9.2f),
        Alumno_nuevo("Paolo",  1002, 7.5f),
        Alumno_nuevo("Julieta", 1003, 8.8f),
        Alumno_nuevo("Jorge", 1004, 6.9f)
    };

    Ordenador *oInt = Ordenador_nuevo(nums, 8, sizeof(int),    comparaInt,
imprimeInt);
    Ordenador *oCar = Ordenador_nuevo(cars, 6, sizeof(char),   comparaChar,
imprimeChar);
    Ordenador *oAlu = Ordenador_nuevo(alu,  4, sizeof(Alumno), Alumno_comparar,
Alumno_imprimir);

    oInt->mostrar(oInt, "Enteros antes:   ");
    oInt->ordenar(oInt);
    oInt->mostrar(oInt, "Enteros despues: ");

    printf("\n");
    oCar->mostrar(oCar, "Chars antes:     ");
    oCar->ordenar(oCar);
    oCar->mostrar(oCar, "Chars despues:   ");

    printf("\n");
    oAlu->mostrar(oAlu, "Alumnos antes:");
    oAlu->ordenar(oAlu);
    oAlu->mostrar(oAlu, "Alumnos despues (por promedio):");

    Ordenador_destruir(oInt);
    Ordenador_destruir(oCar);
    Ordenador_destruir(oAlu);
    return 0;
}
```